UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE TECNOLOGIE DELL'INFORMAZIONE

Documentazione del Progetto di Laboratorio di Sistemi Operativi

Tris

Anno Accademico 2024–2025

Autori:
Simone Di Costanzo - N86004655
Raffaele Raia - N86004564

Indice

1 Introduzione - Requisiti **2**

2 Design del Sistema **3**
2.1 Containerizzazione . . . 3
2.2 Versionamento del Codice . . . 3
2.3 Gestione della Configurazione e Build . . . 3
2.4 Comunicazione . . . 3

3 Backend **4**
3.1 Architettura del Sistema . . . 4
3.1.1 Gestione delle strutture dati condivise . . . 4
3.1.2 Generazione degli ID e opreazioni $O(1)$ . . . 5
3.1.3 Gestione dei comandi principali . . . 5
3.1.4 Gestione della concorrenza dei socket . . . 6
3.2 DynamicArray . . . 6
3.2.1 Struttura dati . . . 6
3.2.2 Caratteristiche principali . . . 6
3.2.3 Funzioni principali . . . 7
3.2.4 Strategia di gestione memoria . . . 7
3.3 Logica del Gioco del Tris . . . 7
3.4 Note Aggiuntive . . . 8

4 Frontend **9**
4.1 Architettura del frontend . . . 9
4.2 Funzionalità dell'interfaccia utente . . . 9
4.3 Integrazione con il client di rete . . . 10
4.4 Logica di Gioco . . . 10
4.4.1 Struttura interna . . . 10
4.4.2 Principali funzionalità . . . 10

1 Introduzione - Requisiti

Il presente progetto ha come obiettivo la realizzazione di un'applicazione client-server in grado di gestire partite del gioco del Tris in modalità multi-client.

L'applicazione è stata sviluppata nell'ambito del corso di Laboratorio di Sistemi Operativi e si basa su un'architettura in cui un server centrale coordina più client connessi, permettendo loro di creare e partecipare a partite.

Un utente può creare una o più partite, ma può partecipare attivamente ad una sola partita per volta. Quando una partita viene creata, viene inizialmente posta nello stato di "ATTESA". In seguito, essa può transitare negli stati di "IN CORSO" o "TERMINATA".

Il creatore della partita ha la possibilità di accettare o rifiutare le richieste di partecipazione ricevute dagli altri giocatori. Una volta accettata una richiesta, la partita entra nello stato "IN CORSO", fino alla sua conclusione.

Al termine della partita, lo stato viene impostato su "TERMINATA" e ciascun giocatore partecipante riceve l'esito, Che può essere "vittoria", "sconfitta" o "pareggio". Qualora la partita finisca in pareggio, è possibile effettuare una rivincita se entrambi i client sono d'accordo.

Questo progetto mira non solo a implementare correttamente la logica del gioco e della comunicazione client-server, ma anche a gestire in maniera efficiente gli stati delle partite e il flusso di interazioni tra i diversi utenti connessi.

2 Design del Sistema

Questa sezione descrive le scelte progettuali relative alla distribuzione, alla gestione del codice e agli strumenti utilizzati per garantire scalabilità, manutenibilità e facilità di sviluppo.

2.1 Containerizzazione

Per semplificare il deployment e garantire la portabilità, il sistema è stato containerizzato utilizzando **Docker**.

Sono stati creati due container distinti:

- **Server**: contiene il server C con tutte le librerie necessarie.
- **Client**: ambiente Java per l'esecuzione del client.

In questo modo è possibile eseguire l'intero sistema su qualunque macchina senza conflitti di dipendenze, assicurando una maggiore riproducibilità degli esperimenti.

2.2 Versionamento del Codice

Il progetto è gestito tramite il sistema di controllo di versione **Git**.

2.3 Gestione della Configurazione e Build

La compilazione e l'esecuzione del progetto sono state automatizzate tramite:

- `Makefile` per il backend C.
- `Maven` per il frontend Java.

2.4 Comunicazione

La comunicazione tra client e server avviene tramite **socket TCP**. Questa scelta è stata fatta per garantire:

- **Affidabilità**: il protocollo TCP assicura consegna ordinata e senza perdite dei messaggi, condizione fondamentale per la coerenza del gioco.
- **Connessione persistente**: una volta stabilita, la connessione resta attiva per l'intera durata della sessione di gioco, evitando overhead di riconnessione ad ogni messaggio.

3 Backend

Il backend è stato realizzato in linguaggio C e rappresenta il cuore logico del sistema.

3.1 Architettura del Sistema

All'avvio, il server inizializza un socket TCP in ascolto sulla porta predefinita o passata come argomento. Per ogni nuovo client che si connette, viene creato un thread dedicato tramite la libreria `pthread`, che si occupa di gestire la comunicazione con quel singolo client. In questo modo, il server è in grado di gestire più client contemporaneamente in maniera concorrente.

Il server mantiene due strutture dati dinamiche per tenere traccia dei client connessi e delle partite in corso. Quando un client si connette, viene aggiunto alla lista dei client e può inviare comandi per creare, partecipare o gestire partite.

Una volta avviato, il server esegue una serie di operazioni fondamentali per la gestione delle connessioni e della logica di gioco. In particolare, il flusso operativo del server è strutturato come segue:

- **Inizializzazione**: creazione del socket TCP e binding alla porta.
- **Ascolto e Accettazione delle connessioni**: il server entra in modalità `listen` per accettare nuove connessioni da parte dei client.
- **Gestione dei client**: per ogni nuova connessione accettata, viene creato un thread dedicato che si occupa della comunicazione con il rispettivo client.
- **Elaborazione delle richieste**: ciascun thread interpreta ed esegue i comandi ricevuti dal client.
- **Deallocazione delle risorse**: alla chiusura della connessione con un client il thread corrispettivo libera le strutture dinamiche per la gestione del client disconnesso.

3.1.1 Gestione delle strutture dati condivise

Il server utilizza due strutture dati principali: una lista dei client connessi e una lista delle partite. Queste strutture sono implementate tramite gli array della libreria `DynamicArray` che permettono operazioni di inserimento, accesso e rimozione in tempo costante, ossia $O(1)$ sfruttando la logica di gestione degli ID.

1: Struttura client

```c
struct client{

    unsigned int clientfd;
    char* name;
    pthread_mutex_t sock_mutex;

};
```

Ogni struttura client ha un mutex **sock_mutex** che blocca l'accesso al socket per le operazioni di invio e ricevimento dei dati che non devono essere interleaved.

2: Struttura game

```c
struct game{

    unsigned short int id_game;
    struct client *client1;
    struct client *client2;
    unsigned int **table;
    game_status status;
    unsigned int turn;

};
```

3.1.2 Generazione degli ID e opreazioni $O(1)$

Sono state progettate le funzioni **generate_id()** e **release_id()** sfruttando anche le caratteristiche della libreria **DynamicArray**, in modo da permettere le operazioni di accesso, rimozione e inserimento in tempo costante $O(1)$.

La funzione di generazione degli id ritorna interi in maniera crescente; quando un ID viene rilasciato con la chiamata alla funzione **release_id()** quest'ultimo viene segnato come disponibile e ritornato alla prossima chiamata di **generate_id()**. In questo modo gli ID possono essere riciclati senza sprechi.

Applicando questa logica all'indicizzazione degli array della libreria **DynamicArray** è possibile far combaciare gli ID con le celle dell'array. Inoltre i "buchi"(eliminazione di un dato ad una determinata cella che non sia l'ultima più a destra) generati dalla rimozione di un game alla sua terminazione o allo scadere del timeout, non sono permanenti poiché le partite hanno una durata limitata massima definita dalla costante TIMEOUT_SEC ([20-30]) e dopo una serie di eliminazioni l'array effettua lo shrink automatico della sua lunghezza e inoltre le celle possono essere riutilizzate alla generazione di una nuova partita.

Pertanto, la presenza temporanea di celle vuote è un compromesso accettabile, giustificato dal vantaggio di garantire operazioni istantanee sull'array.

3.1.3 Gestione dei comandi principali

Il server implementa un sistema di comunicazione command-based: ogni client invia al server un intero (enum command - definito in server.h) che rappresenta un'operazione da eseguire. I comandi vengono ricevuti sul socket associato al client, decodificati e instradati alla funzione corrispondente. Questo approccio permette di centralizzare la logica di gestione ed estendere facilmente il protocollo introducendo nuovi comandi.

```c
typedef enum {
    SERVER_OK = 0,
    CREATE_GAME,
    GET_QUEUE,
    START_GAME,
    DISCONNECT,
    ACCEPT,
    REJECT
} command;
```

- **Avvio Partita (START_GAME)**: il client invia il proprio nome utente e id della partita alla quale vuole partecipare al momento della connessione.

- **Recupero lista partite (GET_QUEUE)**: il client può richiedere la lista delle partite in "ATTESA" e "IN CORSO".
- **Creazione partita (CREATE_GAME)**: il client può avviare una nuova partita, il cui stato viene settatto `IN_ATTESA`.
- **Accettazione o rifiuto**: il creatore della partita riceve la richiesta e può decidere se accettare o rfiutare.
- **Disconnessione (DISCONNECT)**: in qualsiasi momento un client può disconnettersi, tale evento è stato catturato per garantire una deallocazione delle risorse tempestiva.

3.1.4 Gestione della concorrenza dei socket

Per ogni client connesso viene creato un thread dedicato, responsabile della gestione dell'intero ciclo di vita della connessione (ricezione comandi, elaborazione e invio delle risposte). I comandi ricevuti dal client invocano funzioni che possono coinvolgere più client contemporaneamente: in questo modo si evita la necessità di code di messaggi o di altre strutture intermedie, a condizione di gestire correttamente le *race condition*.

Per prevenire conflitti di accesso, ogni struttura `client` possiede un `mutex`. Il thread dedicato al client blocca il mutex per contollare tramite un'operazione di lettura non bloccante, la presenza di nuovi comandi; al termine dell'operazione, il mutex viene sbloccato.

Quando un comando viene effettivamente ricevuto, il mutex è rilasciato dal thread di ricezione e bloccato dal thread che esegue l'operazione richiesta. In questo modo garantiamo che i dati siano letti e scritti solo dal thread che gestisce l'operazione, evitando interferenze con il thread che monitora i comandi.

È importante sottolineare che il thread che esegue l'operazione non è un nuovo thread creato appositamente, ma è il thread del client che ha richiesto l'operazione.

3.2 DynamicArray

La libreria `DynamicArray` implementa una struttura dati equivalente ad un array dinamico thread-safe. Consente l'inserimento, la rimozione e l'accesso a elementi di dimensione arbitraria, con gestione automatica di *espansione* e *riduzione* della capacità allocata.

3.2.1 Struttura dati

```c
typedef struct {
    void  *data;            // area di memoria contenente gli elementi
    size_t single_data_size; // dimensione di un singolo elemento
    size_t size;            // numero di elementi attualmente utilizzati
    size_t capacity;        // numero di slot attualmente allocati
    size_t min_capacity;    // capacita minima sotto la quale non si effettua shrink
    pthread_mutex_t mutex;  // mutex per garantire thread-safety
} DynamicArray;
```

3.2.2 Caratteristiche principali

- **Thread-safety**: tutte le operazioni bloccano un mutex interno, evitando race condition in contesti multithread.

- **Espansione automatica**: quando l'array è pieno, la capacità viene raddoppiata.
- **Riduzione automatica (shrink)**: quando la metà superiore dell'array è vuota, la capacità può essere ridotta (fino a `min_capacity`).
- **Generico**: la struttura è indipendente dal tipo di dato; l'utente specifica la dimensione (`data_size`) in fase di inizializzazione.

3.2.3 Funzioni principali

`int da_init(DynamicArray *da, size_t initial_capacity, size_t data_size)` Inizializza la struttura. Se `initial_capacity==0`, l'array viene allocato al primo inserimento.

`int da_push(DynamicArray *da, const void *elem)` Inserisce un nuovo elemento in coda. Se necessario, raddoppia la capacità.

`int da_pop(DynamicArray *da, void *out_elem)` Rimuove l'ultimo elemento. Se `out_elem != NULL`, copia l'elemento rimosso. Attiva un controllo di shrink.

`int da_insert_at(DynamicArray *da, size_t index, const void *elem)` Inserisce un elemento in una posizione indicata dalla variabile **index**.

`int da_get_at(DynamicArray *da, size_t index, void *out_elem)` Legge l'elemento all'indice specificato. Restituisce errore se l'indice è fuori range.

`int da_remove_at(DynamicArray *da, size_t index, void (*callback)(void *))` Rimuove l'elemento all'indice specificato. Se è fornita una `callback`, questa viene chiamata per liberare risorse associate all'elemento. Lo slot viene azzerato e può attivare lo shrink.

`void da_free(DynamicArray *da, void (*free_callback)(void *))` Libera l'intera struttura. Se è fornita una `callback`, questa viene applicata a ciascun elemento prima della deallocazione.

3.2.4 Strategia di gestione memoria

- La capacità cresce in maniera esponenziale (fattore 2).
- La riduzione è conservativa: avviene solo se la metà superiore dell'array è interamente vuota.
- `min_capacity` impedisce di ridurre troppo la memoria allocata.

3.3 Logica del Gioco del Tris

La partita vera e propria è gestita nella funzione `tris`, che coordina i turni tra i due giocatori (`client1` e `client2`). Il turno iniziale è scelto casualmente con `rand()`, e i giocatori alternano mosse inviando la cella selezionata (da 1 a 9).

La tabella di gioco è un array bidimensionale 3x3 allocato dinamicamente, accessibile tramite un puntatore nella struttura **game**. Dopo ogni mossa, il server verifica la vittoria con funzioni come `check_row`, `check_col` e `check_diag`, o un pareggio se la tabella è piena. Lo stato della partita (`game_status`) transita da `ATTESA` a `INCORSO` e infine a `TERMINATA`.

In caso di pareggio, la funzione `rivincita` propone una nuova partita: crea due thread per leggere le risposte (`ACCEPT` o `REJECT`) da entrambi i client e, se concordi, resetta la tabella con `reset_table` e riavvia `tris`.

La concurrency è garantita bloccando i mutex dei socket durante l'intero gioco per evitare interleaving di messaggi.

3.4 Note Aggiuntive

In aggiunta, il server è stato progettato per gestire in modo robusto:

- Client che si disconnettono improvvisamente.
- Timeout di inattività grazie all'impostazione di un timeout.

Questo garantisce una stabilità capace di offrire un esperienza di spessore.

4 Frontend

Il frontend dell'applicazione è stato sviluppato utilizzando Java e JavaFX, al fine di fornire un'interfaccia utente semplice e intuitiva per l'interazione con il server di gioco. L'applicazione consente all'utente di visualizzare le partite in corso, creare nuove partite, unirsi a partite esistenti e giocare.

4.1 Architettura del frontend

L'architettura del frontend si fonda sulla separazione tra interfaccia grafica, logica di presentazione e comunicazione di rete, garantendo modularità e manutenibilità. I componenti principali sono:

- **Viste (FXML)**: descrivono l'interfaccia grafica (finestre, pulsanti, liste, griglia di gioco). I file `home.fxml` e `game.fxml` corrispondono rispettivamente alla schermata principale e alla schermata della partita.
- **Controller**: gestiscono l'interazione con l'utente. `HomeController` regola la schermata iniziale (connessione al server, creazione e join delle partite), mentre `GameController` coordina il flusso della partita (turni, mosse, aggiornamento della griglia).
- **Logica di dominio**: la classe `TrisGame` racchiude le regole del gioco (validità delle mosse, stato del tabellone, condizioni di vittoria o pareggio).
- **Client di rete**: la classe `GameClient` incapsula la comunicazione con il server, esponendo ai controller metodi di alto livello per gestire le partite e inviare le mosse.

A supporto vi sono due classi di utilità:

- `Home`, entry point dell'applicazione, che carica la vista iniziale e assegna il relativo controller.
- `GameWindow`, che gestisce la transizione dalla schermata iniziale a quella di gioco.

4.2 Funzionalità dell'interfaccia utente

L'interfaccia è dunque organizzata in due viste principali:

- **Schermata iniziale**: consente di visualizzare la lista delle partite disponibili, inserire il proprio nome utente, creare una nuova partita o unirsi a una esistente. Qui compaiono anche notifiche relative alla connessione e agli eventi principali.
- **Schermata di gioco**: mostra la griglia del Tris (3x3) e i dettagli della partita (turno corrente, simboli dei giocatori, esito finale). I pulsanti vengono abilitati o disabilitati in base al turno, e al termine è disponibile una finestra pop-up per proporre o rifiutare una rivincita.

4.3 Integrazione con il client di rete

Il frontend interagisce con il server esclusivamente tramite la classe `GameClient`, che nasconde i dettagli tecnici di socket e protocolli. I controller invocano metodi come:

- `createGame()` per avviare una nuova partita;
- `listGames()` per ottenere la lista delle partite in attesa;
- `startGame()` per unirsi a una partita;
- `sendMove()` per inviare una mossa.

La ricezione dei messaggi è gestita in modo asincrono tramite `startReader()`, che attiva un thread dedicato e notifica la GUI mediante l'interfaccia `GameEvents`, inviando eventi relativi a turno iniziale, mosse avversarie, stato della partita o eventuali errori di connessione.

Questa organizzazione mantiene separata la logica di comunicazione da quella di presentazione, rendendo l'applicazione estendibile e facilitando l'eventuale riuso del client in altri contesti (ad esempio un'interfaccia a riga di comando).

4.4 Logica di Gioco

La logica del gioco del Tris è stata incapsulata nella classe `TrisGame`, la quale gestisce le mosse e verifica le condizioni di vittoria. .

4.4.1 Struttura interna

La classe utilizza un array di 9 celle per rappresentare la griglia 3x3, inoltre mantiene informazioni sul simbolo del giocatore e sul simbolo dell'avversario.

4.4.2 Principali funzionalità

La classe espone divers chei metodi :

- `makeMove(int cellIndex)`: registra la mossa del giocatore nella cella indicata, validando l'input e impedendo sovrascritture.
- `applyOpponentMove(int cellIndex)`: aggiorna la griglia con la mossa ricevuta dall'avversario.
- `isBoardFull()`: verifica se tutte le celle sono occupate (condizione di pareggio).
- `checkWinner()`: controlla le combinazioni vincenti (righe, colonne e diagonali) restituendo l'ID del giocatore vincitore o 0 in caso di partita ancora in corso.